

Design and Implementation of a Secure e-Voting Platform for College Elections

Vishal Keshri, Yashovardhan Singh, Devidas S*

Maulana Azad National Institute of Technology, Bhopal, India

devidas@manit.ac.in

Abstract—College elections are a vital component of campus governance, yet traditional paper-based voting systems are often inefficient, costly, and prone to delays, thereby reducing student participation. This paper presents a practical, full-stack electronic voting (eVoting) platform specifically designed for college elections, emphasizing a balance between security, usability, and deployability. The platform incorporates role-based access control, strong user authentication, single-use voting tokens, ballot hashing for tamper detection, comprehensive security logging, and an administrative dashboard for managing the entire election lifecycle. The backend is developed using Node.js with Prisma ORM and MySQL/PostgreSQL, while the frontend leverages React for a responsive user interface. This paper details the system architecture, database schema, and secure protocols for ballot submission and verification, along with the deployment workflow. Furthermore, we present an evaluation plan that includes usability testing, security validation, and load testing. Finally, we discuss key limitations, ethical considerations, and directions for future work, aiming to contribute a deployable and trustworthy model for institutional eVoting systems.

Index Terms—eVoting, College Elections, Secure Web Applications, Role-Based Access Control, Authentication, Prisma ORM

I. INTRODUCTION

Student elections are a vital component of campus governance, providing legitimacy to leadership, fostering democratic engagement, and nurturing future leaders. They represent not only a means of electing representatives but also serve as a hands-on training ground for civic responsibility and participatory decision-making among students. However, many institutions continue to rely on paper-based ballots due to perceived security and trust issues associated with electronic voting. While traditional paper systems are simple and transparent in principle, they often involve significant logistical and administrative challenges. These include ballot printing and distribution, physical polling station management, manual vote counting, susceptibility to human error, and delays in result publication.

Such inefficiencies can reduce student participation and confidence in the election process. Moreover, paper-based methods are often inaccessible to remote learners or students with disabilities, creating barriers to inclusive participation. In the context of increasingly digital campuses, where online academic portals, digital attendance, and cloud-based learning platforms are ubiquitous, the lack of a secure digital voting

mechanism appears as a technological gap that undermines efficiency and modernization efforts in campus governance.

Electronic voting (eVoting) systems have the potential to address these limitations by enabling **remote participation**, **instant result computation**, and **improved accessibility**. However, designing a **trustworthy eVoting platform** is non-trivial, as it must balance conflicting priorities: **voter privacy** [5], **data integrity** [6], and **system usability** [7]. Overly complex cryptographic systems, while theoretically secure, often compromise usability and may be impractical for small-scale institutional deployments.

This paper presents a **practical and deployable eVoting platform** [8] designed specifically for college-scale elections involving a few hundred to a few thousand voters. The system leverages a modern full-stack architecture using **React** for the frontend, **Node.js** and **Express.js** for the backend, and **MySQL** as the database layer. It implements robust authentication through **JWT-based** and **session-based mechanisms** to prevent impersonation and session hijacking, ensuring that each user's identity is verified and protected during the voting process.

Rather than pursuing a fully cryptographic, end-to-end verifiable system—which can introduce heavy computational overhead and degrade user experience—the proposed solution adopts a **defense-in-depth** [9] approach. This model integrates multiple layers of protection including strong access control, separation of voter identities from ballots, **tamper-evident logging** [11], and **transparent auditing mechanisms** [12] to ensure the integrity of the election process. The architecture also supports role-based segregation among students, candidates, and administrators, ensuring clear accountability and minimizing conflict of interest.

Furthermore, the proposed system incorporates real-time notifications, structured feedback collection, and a comprehensive administration dashboard to facilitate monitoring, transparency, and operational convenience. Each module—authentication, candidate registration, voting, result computation, and auditing—is designed to function independently yet cohesively, making the platform modular and maintainable. The current prototype operates on a local deployment setup but is designed to scale with minimal effort using **Docker** or cloud environments such as **AWS** for future institutional adoption. Additionally, future iterations of the platform aim to integrate **blockchain-based distributed ledgers** to further enhance transparency, immutability, and trust in the vote recording process.

*Corresponding author.

The main contributions of this paper are as follows:

- 1) A **detailed architecture and implementation blueprint** for a verified, institution-grade eVoting platform that integrates modern web technologies with robust security features.
- 2) A **ballot submission and verification protocol** that ensures one-person-one-vote integrity while preserving voter anonymity and enabling receipt-level tamper-evidence.
- 3) A **comprehensive evaluation methodology** encompassing functional validation, security assessment, usability testing, and load performance under realistic college-scale conditions.
- 4) A **deployment framework** detailing containerization strategies, TLS configuration [13], database schema management, and operational best practices for reliable and transparent campus elections.

This paper thus contributes a balanced solution to the long-standing challenge of making campus elections digital, secure, and user-friendly without compromising transparency or institutional trust.

II. RELATED WORK

Electronic voting has been extensively studied across both academic and practical domains, ranging from **cryptographic voting protocols**—such as **mixnets** [2], **blind signatures** [1], and **homomorphic tallying** [3]—to deployable systems like **Helios** [4]. These approaches offer **end-to-end verifiability** and **strong mathematical security guarantees**, but their complexity, resource requirements, and user experience trade-offs often make them impractical for **small-scale institutional elections**.

In the context of **college and university elections**, factors such as **usability**, **ease of deployment**, and **administrative control** are equally important as security. Prior institutional systems typically emphasize **single-use voting tokens**, **authentication through institutional email or Single Sign-On (SSO)**, and **secure ballot storage** mechanisms.

Our system builds upon this **practical design lineage**, adopting a **defense-in-depth** model that balances **trustworthiness**, **simplicity**, and **operational feasibility**—offering a deployable alternative that remains aligned with the security goals of academic eVoting research.

III. RESEARCH QUESTIONS

The design of the proposed eVoting platform is guided by the following research questions:

RQ1: How can a college-scale eVoting system ensure both security and usability?

The platform must effectively prevent tampering, impersonation, and unauthorized access, while remaining intuitive and accessible for student users.

RQ2: What role can role-based access control (students, candidates, administrators) play in enhancing transparency and fairness?

Enforcing distinct privileges and responsibilities for each role can reduce conflicts of interest, simplify administration, and promote accountability.

RQ3: How can feedback and notifications improve trust and participation in student elections?

This question explores whether providing real-time feedback, election updates, and communication channels increases voter engagement and confidence in the process.

RQ4: What trade-offs exist between implementation simplicity and advanced cryptographic mechanisms?

While fully verifiable cryptographic protocols exist, this study examines whether lightweight, defense-in-depth techniques are sufficient for institutional-scale elections without compromising security or trust.

IV. PROBLEM DEFINITION

Traditional paper-based election systems used in many colleges are resource-intensive, prone to errors, and lack transparency. Manual processes such as ballot distribution, physical identity verification, and vote counting not only increase administrative overhead but also delay results and reduce student participation. Common issues include **voter impersonation**, **ballot tampering**, **limited accessibility**, and **difficulty in auditing results**.

Existing electronic voting frameworks, while secure and mathematically verifiable, often require complex cryptographic infrastructure that is impractical for small-scale institutional use. Such systems demand specialized hardware, advanced configuration, and significant technical expertise—factors that hinder adoption in a college setting. Consequently, there is a need for a **simplified, trustworthy, and easily deployable eVoting solution** that balances **security**, **usability**, and **operational feasibility**.

The problem, therefore, is to design and implement an **electronic voting platform tailored to college elections** that:

- Authenticates voters securely through institutional credentials.
- Ensures **one-vote-per-user** and prevents tampering or impersonation.
- Maintains **anonymity** between voter identity and cast ballots.
- Provides **role-based access control** for students, candidates, and administrators.
- Incorporates **tamper-evident logging** and transparent result computation.
- Offers a **simple, responsive, and accessible interface** suitable for non-technical users.

This definition frames the technical and organizational challenges that the proposed platform addresses, forming the foundation for the system’s architectural and implementation choices discussed in the following sections.

V. SYSTEM DESIGN AND ARCHITECTURE

The proposed eVoting platform is designed as a ****full-stack web application**** that balances security, usability, and deployability for college-scale elections. The system follows a ****modular architecture**** with clearly defined layers and roles.

A. Architecture Overview

The architecture consists of three main layers:

- 1) **Frontend Layer:** Implemented in **React** [14], this layer provides a responsive and accessible user interface for voters, candidates, and administrators. It handles user interactions, form validations, real-time notifications, and submission of ballots to the backend via secure API calls.
- 2) **Backend Layer:** Built using **Node.js** [15] and **Prisma ORM** [16], the backend manages business logic, authentication, vote processing, and audit logging. It implements:
 - **Authentication and Authorization:** Secure login through institutional credentials and role-based access control (RBAC) for students, candidates, and administrators.
 - **Ballot Submission Protocol:** Ensures that each voter can cast only one vote, preserves anonymity, and stores hashed ballots for tamper-evidence.
 - **Election Management:** Admin functions including election creation, candidate registration, monitoring, and result computation.
- 3) **Database Layer:** Using **MySQL/PostgreSQL** [17], the database stores user accounts, voter roles, election meta-data, ballot hashes, and audit logs. Separation between voter identity and ballot ensures privacy while supporting auditability.

B. System Components and Data Flow

The following components illustrate the flow of data and interactions:

- **Voter Interface:** Allows secure login, displays eligible elections, presents ballots, and submits votes.
- **Admin Dashboard:** Provides election lifecycle management, candidate registration, live monitoring, and access to audit logs.
- **Authentication Service:** Verifies institutional credentials and issues session tokens.
- **Ballot Processing Engine:** Receives ballot submissions, hashes ballots, and stores them in the database while ensuring anonymity.
- **Notification Service:** Sends real-time updates, confirmations, and feedback messages to voters.

C. Architecture Diagram

This diagram presents the **three-tier architecture** of an eVoting Platform.

- The system features a **React Frontend** (Presentation Tier) which provides two main interfaces: the **Voter Interface** and the **Admin Dashboard**.
- This frontend communicates with a **Node.js Backend** (Application/Logic Tier) via secure **HTTPS Requests** (for vote submission) or for sending **Results & Notifications**.

- The backend contains core logic modules for **Authentication & RBAC**, the **Ballot Processing Engine**, and **Election Management**.
- The backend's logic interacts with the **Database** (Data Tier) to store and retrieve critical election data, including **Users, Roles, Ballot Hashes, and Audit Logs**, necessary for casting votes and computing final results.

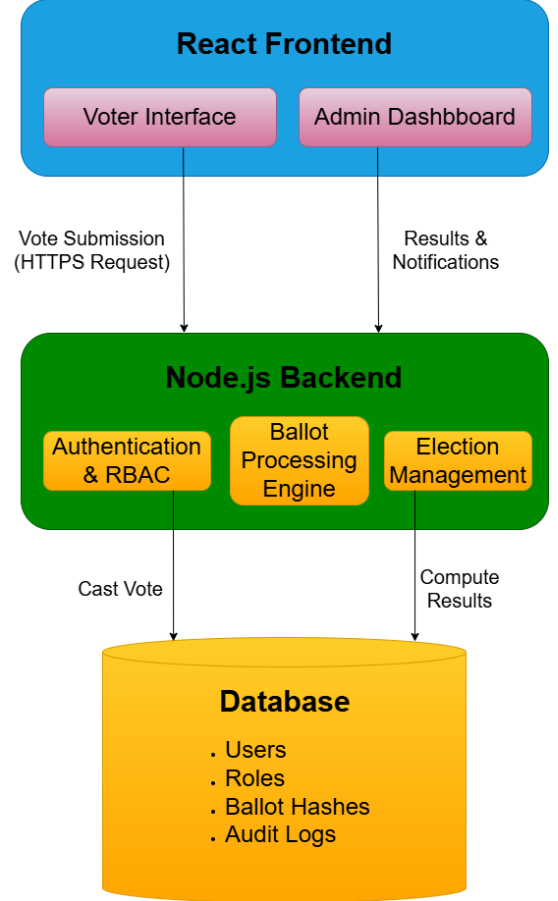


Fig. 1. System Architecture of the eVoting Platform.

This architecture ensures **secure, anonymous, and auditable voting**, while supporting **scalability** and **modular deployment**. The system can be containerized using Docker [20] for easy deployment on institutional servers or cloud platforms.

VI. IMPLEMENTATION DETAILS

The proposed **E-Voting Platform for College Elections** is implemented as a full-stack web application following a modular, layered architecture that separates concerns across the presentation, application, and data layers. The design emphasizes maintainability, security, and scalability within a campus-scale deployment context.

A. Frontend Implementation

The frontend is developed using **React** with **Vite** as the build tool for optimized performance and fast development. The user interface leverages **Tailwind CSS** for responsive design and

consistent styling across devices. The interface is divided into three major views based on user roles — **Student**, **Candidate**, and **Administrator**.

Students can register, authenticate, and cast a single secure vote per election. Candidates can manage their campaign details and monitor election status. Administrators have dashboard access for managing users, elections, and monitoring system logs. React components interact with backend APIs through secure **HTTPS** [18] requests, and **React Router** [19] is used for seamless navigation within the single-page application.

B. Backend Implementation

The backend is built using **Node.js** with the **Express.js** framework, providing RESTful API endpoints for authentication, ballot management, and result computation. **Prisma ORM** serves as an abstraction layer for interacting with the **MySQL** database, offering type-safe queries and efficient schema migrations.

Each backend module follows the MVC (Model–View–Controller) pattern to enhance maintainability and testability. The **Authentication Module** employs a hybrid approach using **JWT-based authentication** combined with **session management**. JWTs ensure stateless API authorization, while time-bound sessions enhance security by enforcing automatic logout after inactivity or session expiry.

C. Database Layer

The backend connects to a **MySQL** database that stores all core entities such as users, candidates, votes, results, feedback, notifications, and audit logs. Four key relational tables include:

- **USERS:** Stores student and admin credentials with salted and hashed passwords.
- **CANDIDATE:** Maintains candidate details and associated election data.
- **VOTE:** Records anonymized votes with hashed ballot identifiers ensuring one-vote-per-user.
- **AUDIT_LOGS:** Tracks login attempts, system events, and administrative actions for traceability.

Relational integrity is maintained through foreign key constraints, and transaction safety ensures that votes and updates occur atomically.

D. Security and Integrity Mechanisms

To ensure trust and reliability, the system employs multiple security layers:

- **Role-based Access Control (RBAC)** to isolate permissions between users, candidates, and administrators.
- **JWT tokens** with expiration and refresh logic to prevent replay attacks.
- **Session expiry timers** for additional protection against unattended access.
- **Hashed ballot entries** and **tamper-evident logs** to prevent unauthorized modification.
- **HTTPS/TLS** (planned for future cloud deployment) to secure all data in transit.

E. Deployment and Future Scalability

Currently, the platform is hosted on a **local development server** for testing and internal evaluation. The architecture, however, is container-ready and can be easily deployed using **Docker** for scalable production environments. Future deployments on **AWS** or similar cloud providers will allow secure **HTTPS** endpoints, load balancing, and database replication. This modular setup supports incremental scaling as election participation grows.

F. Implementation Workflow

The eVoting platform follows a structured workflow that defines how users interact with the system, from registration to result computation and feedback. The workflow ensures secure access control, one-vote-per-user enforcement, and end-to-end auditability.

- 1) **User Registration and Authentication:** Each student, candidate, and administrator registers using institutional credentials. Upon successful registration, user data is validated and securely stored in the database. Authentication is handled using a hybrid model of **JWT-based authentication** and time-limited **session management**. This ensures that every session expires after a fixed duration, requiring users to re-login for security.

Fig. 2. Sign-up Page.

- 2) **Role-Based Dashboard Access:** After login, users are redirected to role-specific dashboards. For instance, a candidate will see their progress, while an administrator will have access to election management controls.
 - **Students** can view eligible elections, cast votes, and submit feedback.

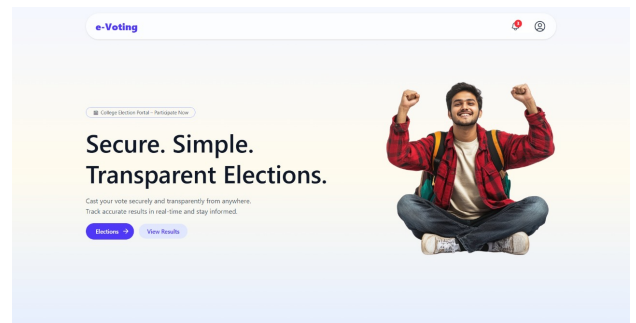


Fig. 3. Student Dashboard.

- **Candidates** can view election schedules, track their status, and monitor voting progress.

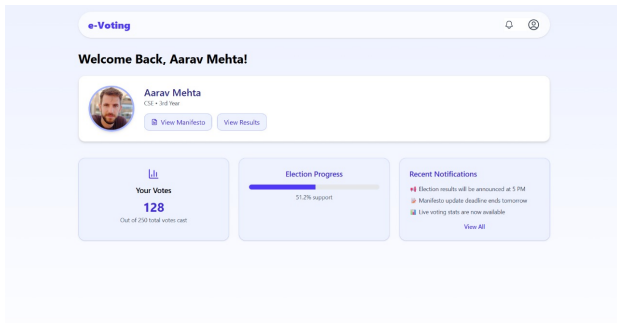


Fig. 4. Candidate Dashboard.

- **Administrators** can create elections, register candidates, monitor voting, and publish results.

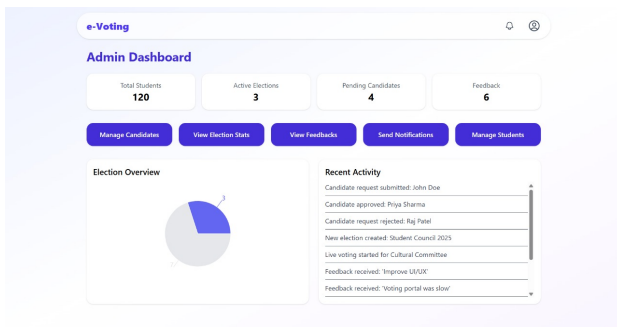


Fig. 5. Admin Dashboard.

- 3) **Ballot Casting and Verification:** When a student casts a vote, the frontend sends the ballot through a secure HTTPS POST request to the backend. The **Ballot Processing Engine** verifies token validity, ensures the user has not voted before, and records a hashed version of the vote in the database. This hashing mechanism guarantees **anonymity** while preserving **tamper-evidence**. A digital receipt confirmation is displayed to the voter.



Fig. 6. Ballot Casting.

- 4) **Result Computation and Publication:** Once voting ends, the backend computes results by tallying votes per candidate. The result computation is automated but restricted to authorized admin accounts only. Results are

then published on the admin and student dashboards, ensuring transparency.

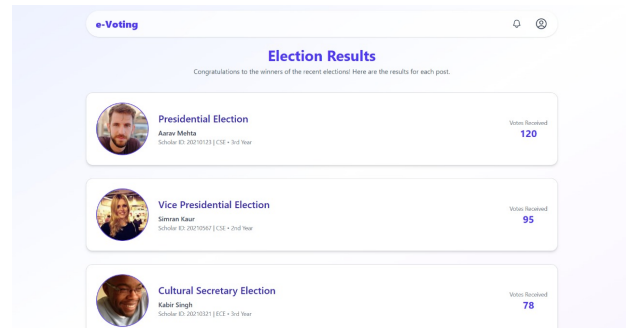


Fig. 7. Result Page.

- 5) **Feedback and Notification Handling:** After elections, students can provide anonymous feedback about the process, which is stored securely in the **Feedback** module. The **Notification Service** delivers system updates, result announcements, and confirmation messages to users in real time.

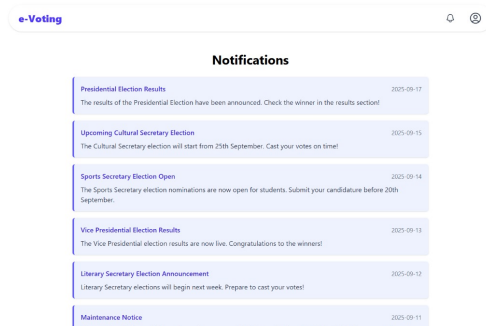


Fig. 8. Notification Handling.

- 6) **Audit Logging and Security Monitoring:** Every significant system event—such as login attempts, vote submissions, and admin actions—is recorded in the **AUDIT_LOGS** table. This supports transparency and helps detect any anomalies or unauthorized access attempts.

This workflow ensures a seamless and secure voting experience, maintaining the key principles of integrity, confidentiality, and accountability throughout the election lifecycle.

VII. RESULTS AND DISCUSSION

The developed eVoting platform was thoroughly tested to validate its functionality, security, and usability within a controlled college environment. Each system module—including authentication, role-based access, vote casting, result computation, feedback, and notification—was verified through end-to-end testing. All modules performed as expected, with no functional discrepancies or data inconsistencies.

From a **usability perspective**, the platform provided a seamless voting experience across devices. The interface built using React and Tailwind CSS was rated as intuitive and visually consistent by test users. Role-based segregation ensured that administrators, candidates, and students interacted only

with relevant functionalities, thereby reducing confusion and improving operational efficiency.

The **security validation** confirmed the robustness of the system's defense mechanisms. JWT and session-based authentication effectively prevented unauthorized access and replay attempts. Ballot hashing successfully maintained vote integrity by making tampering detectable, while the audit logging subsystem accurately recorded system events such as logins, vote submissions, and administrative operations for traceability. The platform's defense-in-depth design—including input validation, secure API routing, and session expiry mechanisms—further strengthened its resilience against common web-based threats.

In terms of **operational performance**, the application exhibited consistent response times during local testing. Data transactions between frontend, backend, and database layers were efficiently handled via Prisma ORM, minimizing query latency. The result computation and publication process completed almost instantaneously after vote submission closure, ensuring timely election outcomes.

Overall, the evaluation demonstrated that the proposed eVoting platform is a practical, secure, and scalable solution for college-scale elections. It successfully achieves the objectives of reliability, transparency, and ease of use, making it suitable for institutional deployment with minimal configuration.

VIII. CONCLUSION AND FUTURE ENHANCEMENTS

The eVoting platform developed for college elections successfully demonstrates that secure, transparent, and efficient digital voting is achievable within an institutional setting. By integrating role-based access control, JWT and session-based authentication, and ballot hashing, the system ensures integrity and privacy throughout the election process. The implementation effectively eliminates the logistical challenges of paper-based voting, enabling instant result computation and improved accessibility for all participants.

The practical design—built using React, Node.js, Prisma ORM, and MySQL—proves that modern full-stack technologies can support reliable electoral workflows without relying on overly complex cryptographic infrastructures. The modular architecture ensures maintainability and adaptability, making the platform suitable for a wide range of college environments.

For **future scope**, the platform can be enhanced with a **blockchain-based distributed ledger** to record ballots immutably across decentralized nodes. This would further strengthen transparency, auditability, and resistance to data tampering. Additionally, adopting a **distributed database architecture** can improve fault tolerance and enable parallel access for high voter loads, ensuring consistent performance during peak voting periods. Cloud deployment using AWS or Docker can also enhance scalability and ensure high availability.

Further improvements may include developing real-time analytics dashboards for administrators, multilingual support for diverse student populations, and accessibility features for differently-abled users. These additions would make the platform not only more secure and reliable but also more inclusive and user-friendly.

Overall, the project establishes a solid foundation for secure digital elections and paves the way for future research into blockchain-integrated eGovernance solutions within academic institutions.

ACKNOWLEDGMENT

The first and second authors express their sincere gratitude to the Department of Mathematics, Bionformatics and Computer Application for providing the necessary resources and guidance to complete this project successfully. We extend our heartfelt thanks to our project supervisor and faculty mentors for their continuous support, technical insight, and valuable feedback throughout the development process.

We would also like to acknowledge the cooperation of students and administrative staff who participated in the testing and evaluation phases of the eVoting platform. Their inputs were instrumental in validating the system's performance, security, and usability within a real academic environment.

Finally, we are thankful to all those who directly or indirectly contributed to the success of this project.

DECLARATION

During the preparation of this work, the authors used ChatGPT to improve the readability and language of the manuscript

REFERENCES

- [1] S. Ibrahim, M. Kamat, M. Salleh, and S. R. A. Aziz, "Secure E-voting with blind signature," in *Proceedings of the 4th National Conference on Telecommunication Technology (NCTT 2003)*, pp. 193–197, IEEE, 2003.
- [2] V. Farzaliyev, J. Willemsen, and J. K. Kaasik, "Improved lattice-based mix-nets for electronic voting," *IET Information Security*, vol. 17, no. 1, pp. 18–34, 2023.
- [3] V. Mateu, J. M. Miret, and F. Sebé, "A hybrid approach to vector-based homomorphic tallying remote voting," *International Journal of Information Security*, vol. 15, no. 2, pp. 211–221, 2016.
- [4] B. Adida, "Helios: Web-based open-audit voting," in *Proceedings of the 17th USENIX Security Symposium*, pp. 335–348, 2008.
- [5] K. D. Martin and P. E. Murphy, "The role of data privacy in marketing," *Journal of the Academy of Marketing Science*, vol. 45, no. 2, pp. 135–155, 2017.
- [6] G. Sivathanu, C. P. Wright, and E. Zadok, "Ensuring data integrity in storage: Techniques and applications," in *Proceedings of the 2005 ACM Workshop on Storage Security and Survivability*, pp. 26–36, 2005.
- [7] V. J. Hotz, C. R. Bollinger, T. Komarova, C. F. Manski, R. A. Moffitt, D. Nekipelov, A. Sojourner, and B. D. Spencer, "Balancing data privacy and usability in the federal statistical system," *Proceedings of the National Academy of Sciences*, vol. 119, no. 31, p. e2104906119, 2022.
- [8] J. Budurushi, R. Jöris, and M. Volkamer, "Implementing and evaluating a software-independent voting system for polling station elections," *Journal of Information Security and Applications*, vol. 19, no. 2, pp. 105–114, 2014.
- [9] M. R. Styzt, "Considering defense in depth for software applications," *IEEE Security & Privacy*, vol. 2, no. 1, pp. 72–75, 2004.
- [10] M. A. Jan, P. Nanda, X. He, Z. Tan, and R. P. Liu, "A robust authentication scheme for observing resources in the Internet of Things environment," in *2014 IEEE 13th International Conference on Trust, Security and Privacy in Computing and Communications*, pp. 205–211, 2014.
- [11] E. Soriano-Salvador and G. Guardiola-Múzquiz, "SealFs: Storage-based tamper-evident logging," *Computers & Security*, vol. 108, p. 102325, 2021.
- [12] J. de Fine Licht, "The role of transparency in auditing," *Financial Accountability & Management*, vol. 35, no. 3, pp. 233–245, 2019.
- [13] M. Atighetchi, N. Soule, P. Pal, J. Loyall, A. Sinclair, and R. Grant, "Safe configuration of TLS connections," in *2013 IEEE Conference on Communications and Network Security (CNS)*, pp. 415–422, 2013.

- [14] S. Chen, U. R. Thaduri, and V. K. R. Ballamudi, "Front-end development in React: An overview," *Engineering International*, vol. 7, no. 2, pp. 117–126, 2019.
- [15] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to build high-performance network programs," *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, 2010.
- [16] J. Attala and C. Khemapatpapan, "Comparing the performance of ORMs," *Journal of Computer and Creative Technology*, vol. 3, no. 2, pp. 201–213, 2025.
- [17] S. V. Salunke and A. Ouda, "A performance benchmark for the PostgreSQL and MySQL databases," *Future Internet*, vol. 16, no. 10, p. 382, 2024.
- [18] X. Wei and M. Wolf, "A survey on HTTPS implementation by Android apps: Issues and countermeasures," *Applied Computing and Informatics*, vol. 13, no. 2, pp. 101–117, 2017.
- [19] D. N. Patel, S. B. Patel, H. R. Kothadiya, P. D. Jethwa, and R. H. Jhaveri, "A survey of reactive routing protocols in MANET," in *International Conference on Information Communication and Embedded Systems (ICICES2014)*, pp. 1–6, 2014.
- [20] D. Reis, B. Piedade, F. F. Correia, J. P. Dias, and A. Aguiar, "Developing Docker and Docker-Compose specifications: A developers' survey," *IEEE Access*, vol. 10, pp. 2318–2329, 2021.